

```
!nvidia-smi
```

```
Sat May 16 19:11:33 2026
```

NVIDIA-SMI 580.82.07			Driver Version: 580.82.07			CUDA Version: 13.0		
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	MIG M.	
0	Tesla T4	Off	00000000:00:04.0	Off				0
N/A	44C	P8	11W / 70W	0MiB / 15360MiB	0%	Default		N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
	ID	ID				Usage	
No running processes found							

```
%%writefile matmul.cu

#include <stdio.h>
#include <cuda_runtime.h>
__global__ void matmul(float* A, float* B, float* C, int N)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if(row < N && col < N)
    {
        float s = 0;
        for(int k = 0; k < N; k++)
        {
            s += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = s;
    }
}

int main()
{
    int N = 2;
    size_t size = N * N * sizeof(float);
    float A[] = {1,2,3,4};
    float B[] = {5,6,7,8};
    float C[4];
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);
    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice); //copy I/P vect from host
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);
    dim3 blocks(N, N);
    dim3 threads(1,1);
    matmul<<<blocks, threads>>>(d_A, d_B, d_C, N);
    cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost); //copy result back from device
    printf("Result Matrix:\n");
    for(int i = 0; i < N*N; i++)
    { printf("%f ", C[i]); }
    cudaFree(d_A); //frees the memory on the device
    cudaFree(d_B);
    cudaFree(d_C);
    return 0;
}
```

```
Writing matmul.cu
```

```
!nvcc matmul.cu -o matmul
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release
```

```
!./matmul
```



Result Matrix:

19.000000 22.000000 43.000000 50.000000